

Deep Learning-Based Image Classification on CIFAR10 Dataset

Evaluating Custom CNNs vs. Transfer Learning (MobileNetV2)

Jawad ALHARAKE

14/07/2026

Project Objective

The primary goal of this project is to comparatively analyze two distinct approaches to deep learning image classification using the CIFAR-10 dataset:



Baseline Establishment: Build, train, and optimize a Custom Convolutional Neural Network (CNN) entirely from scratch to understand foundational feature extraction.



Advanced Implementation: Implement Transfer Learning using a pre-trained MobileNetV2 backbone, modifying its classification head for our specific 10-class task.



Comparative Analysis: Evaluate both models based on accuracy, loss stabilization, computational efficiency (CPU training time), and resistance to overfitting.

The Challenge: CIFAR-10 Dataset

The CIFAR-10 dataset presents a unique challenge for computer vision models due to its structure:

- Contains 60,000 color images divided into 10 mutually exclusive classes (e.g., cars, birds, cats, ships).
- Low Resolution (32x32): Because the images are extremely small, models cannot rely on high-definition textures and must instead learn to classify based on coarse, granular shapes and silhouettes.



- A visual snapshot of a preprocessed training batch sample of the CIFAR10 dataset showcasing different target classes.
- Data augmentation strategy: Presence of slight shifts and black padding boundaries on the edges of the images (like the horse, bird, and airplane) --> artificially enhance dataset variety to prevent overfitting.

```
train_transform = transforms.Compose([transforms.RandomCrop(32, padding=4)])
```

Dataset Distribution & Split Strategy

Total Data Pool: 60,000 total images



1. Training Set (66.6% | 40,000 Images): The "study material" for the networks.

Used directly by the models to adjust internal weights, learn edge detection, and map out class-specific geometries.



2. Validation Set (16.6% | 10,000 Images): The "practice exam" run automatically at the end of every single epoch.

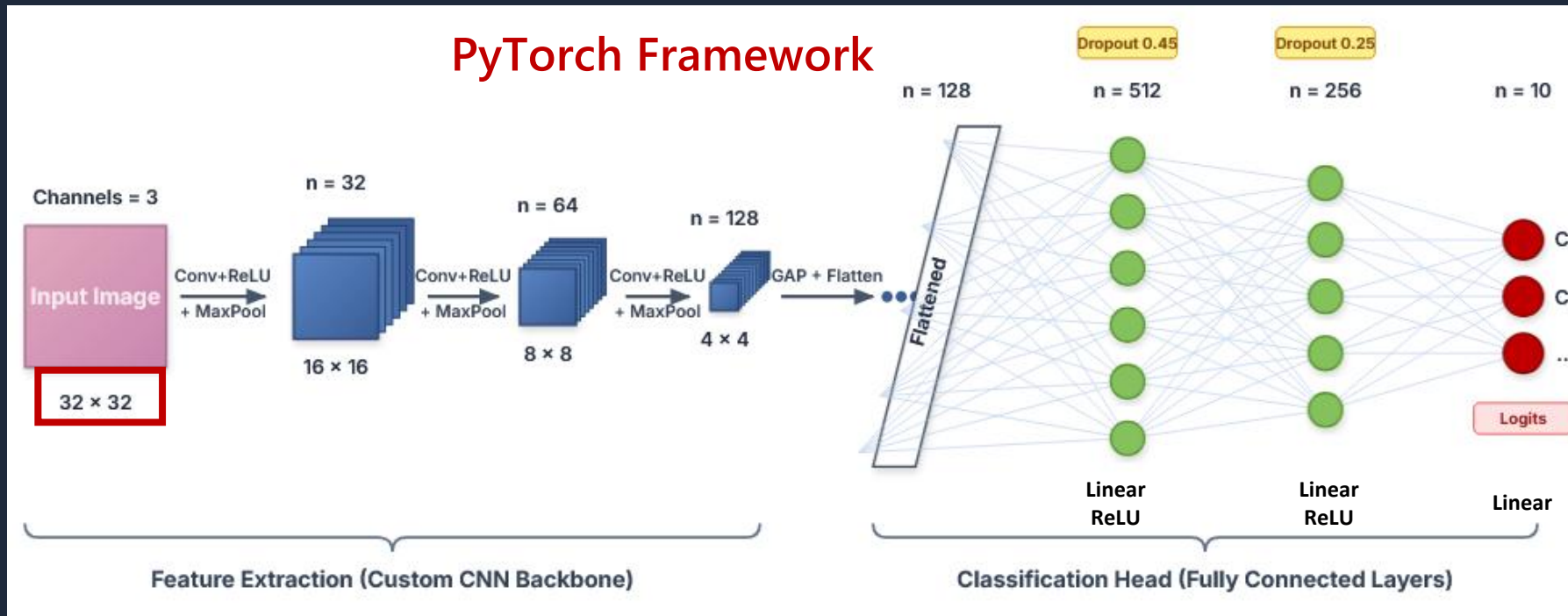
Provides an unbiased look at model generalization. This is the data that the Early Stopping mechanism monitors to detect and prevent overfitting.



3. Test Set (16.6% | 10,000 Images): The final "unseen exam" locked away until training is entirely finished.

Never used to tweak the models; used strictly to calculate the final accuracy metrics and render the final confusion matrices.

Custom CNN: Architecture & Dimensions



```
self.classifier=nn.Sequential(  
    # Fully Connected Layer 1  
    nn.Linear(128, 512),  
    nn.ReLU(inplace=True),  
    nn.Dropout(p=0.45),  
  
    # Fully Connected Layer 2  
    nn.Linear(512, 256),  
    nn.ReLU(inplace=True),  
    nn.Dropout(p=0.25),  
  
    # Final Output Layer  
    nn.Linear(256, num_classes)  
)
```

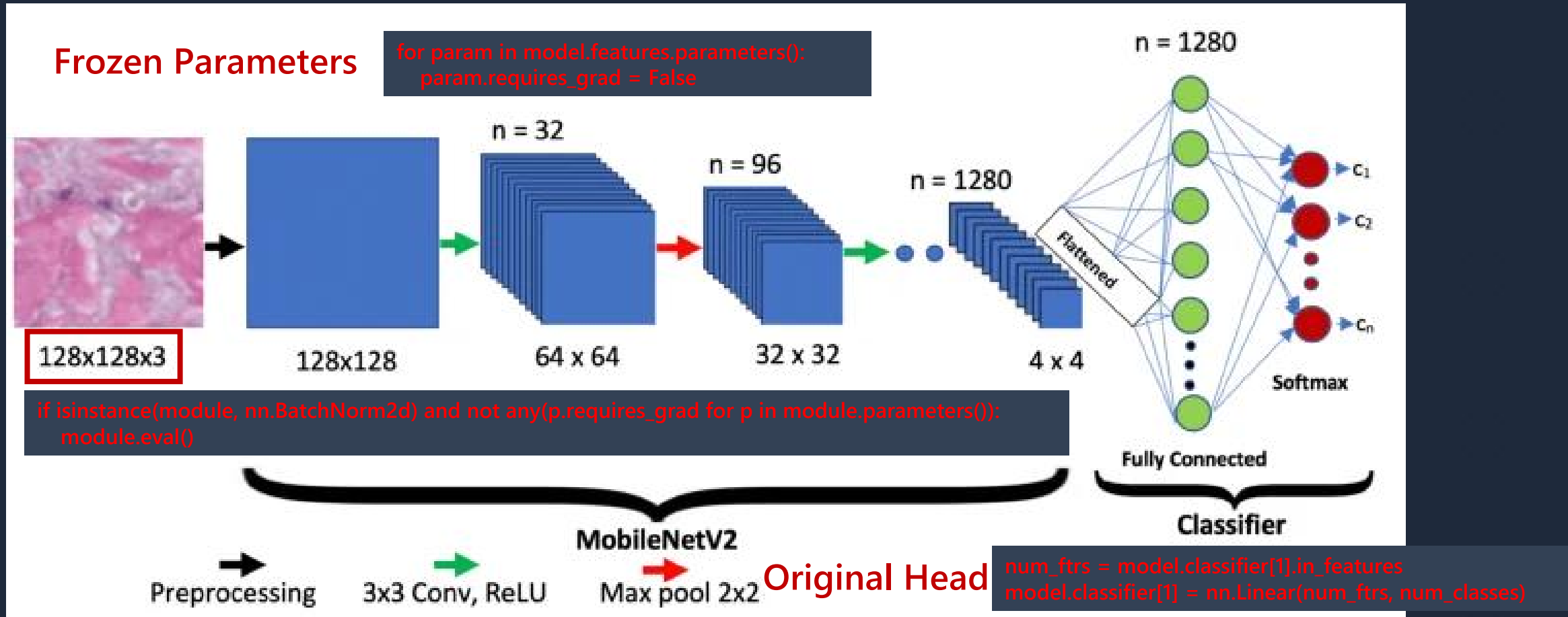
Optimization (Adam)

Uses Adam optimizer for adaptive learning rates per parameter, ensuring faster convergence than standard SGD.

Output Activation (Logits)

No explicit softmax is added to the final layer. Unlike Keras, PyTorch handles the Softmax math internally within `nn.CrossEntropyLoss` for numerical stability.

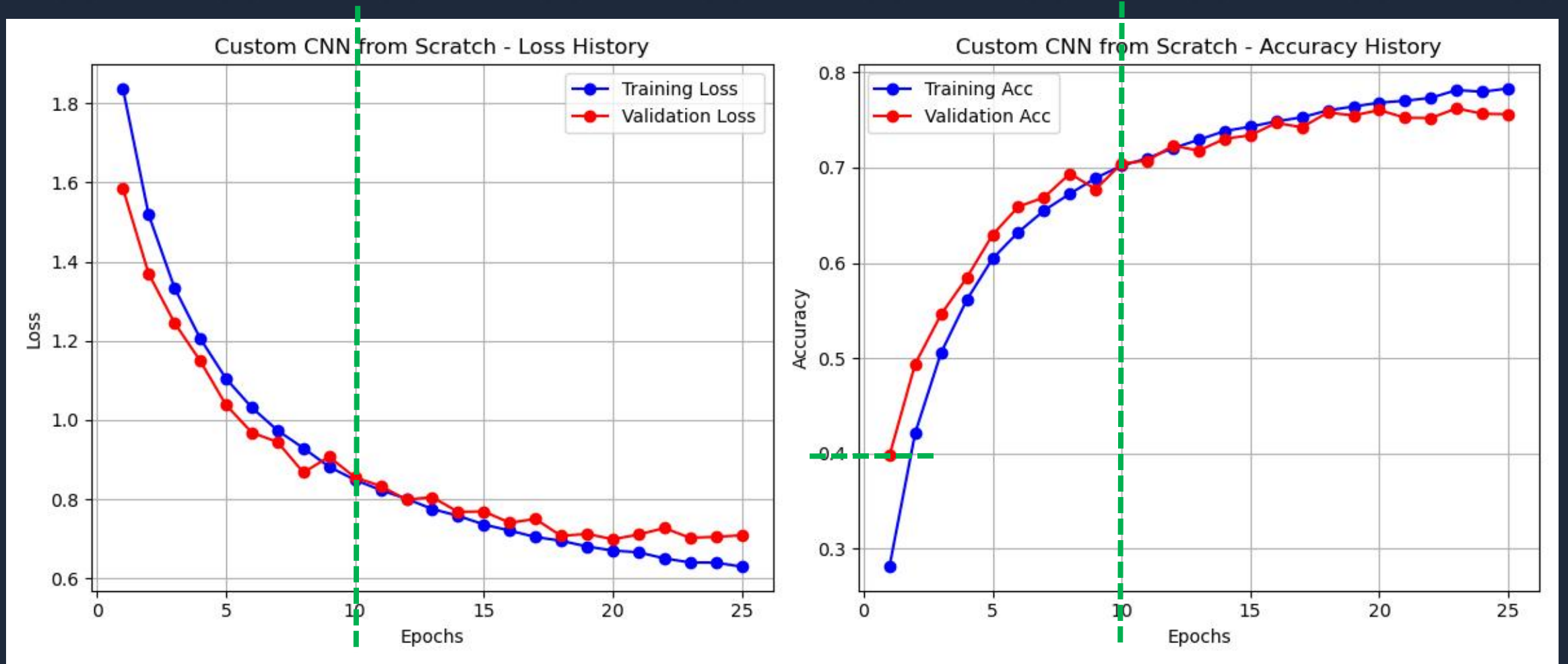
Transfer Learning: MobileNetV2



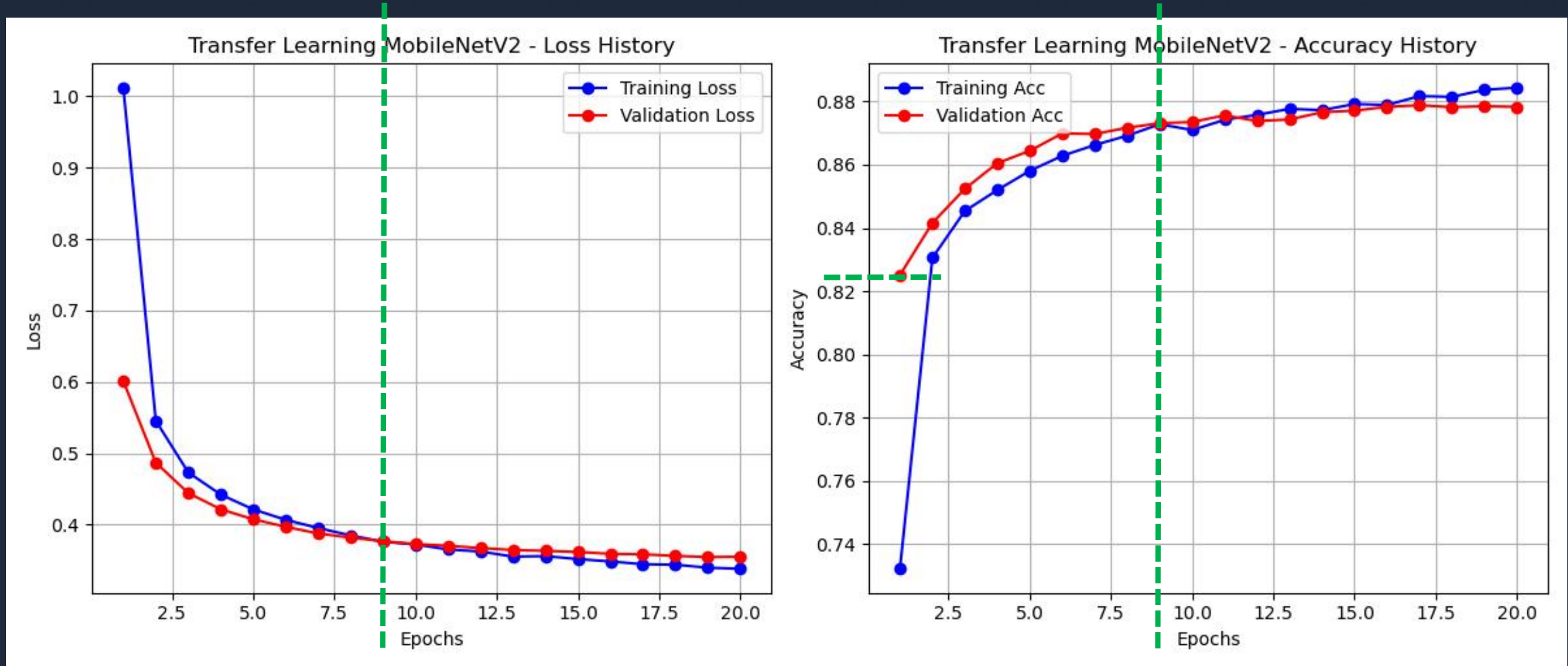
Number of Parameters used

Parameter Category	Custom CNN (From Scratch)	MobileNetV2 (Transfer Learning)
Original Full Model	N/A	3,504,872
Removed Original Head	N/A	-1,281,000
Frozen Parameters	0	2,223,872
Trainable Parameters	293,194	12,810
Total Active Parameters	293,194	2,236,682

Training & Validation Sets - Custom CNN



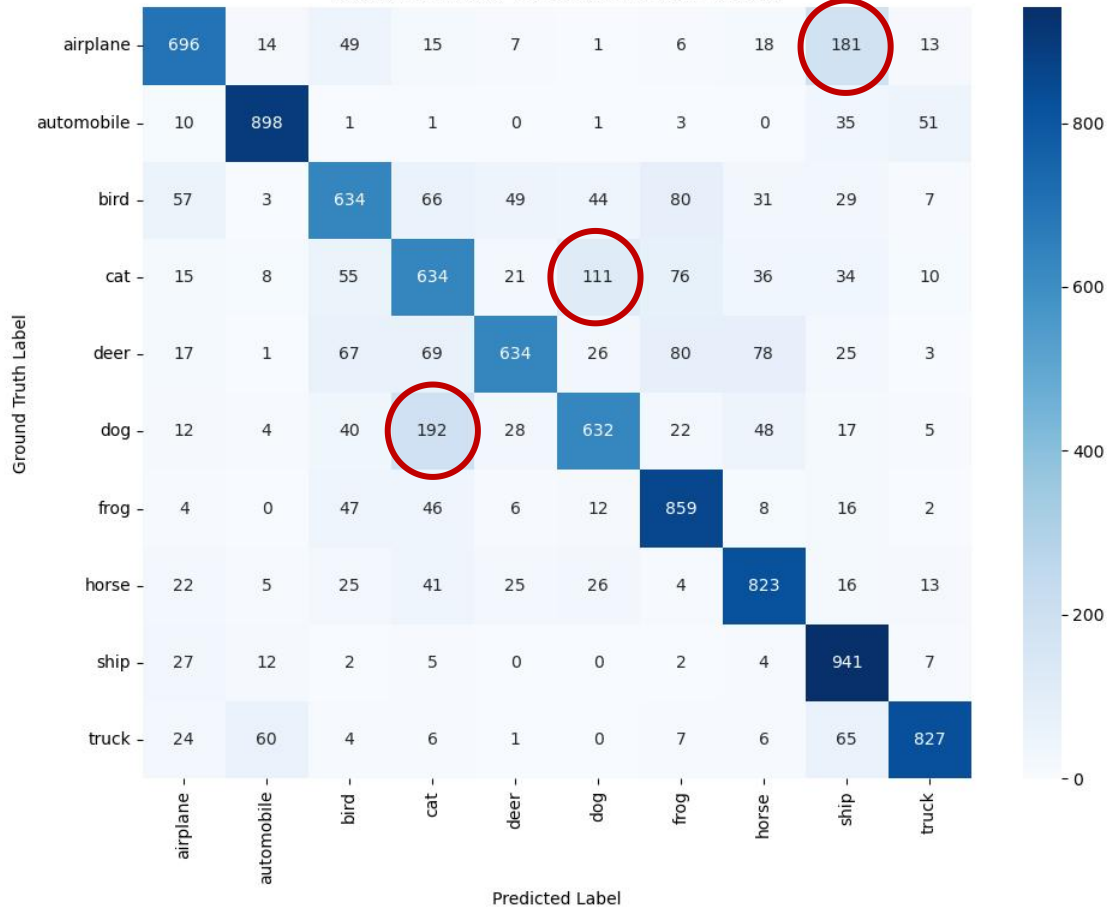
Training & Validation Sets - Transfer Learning



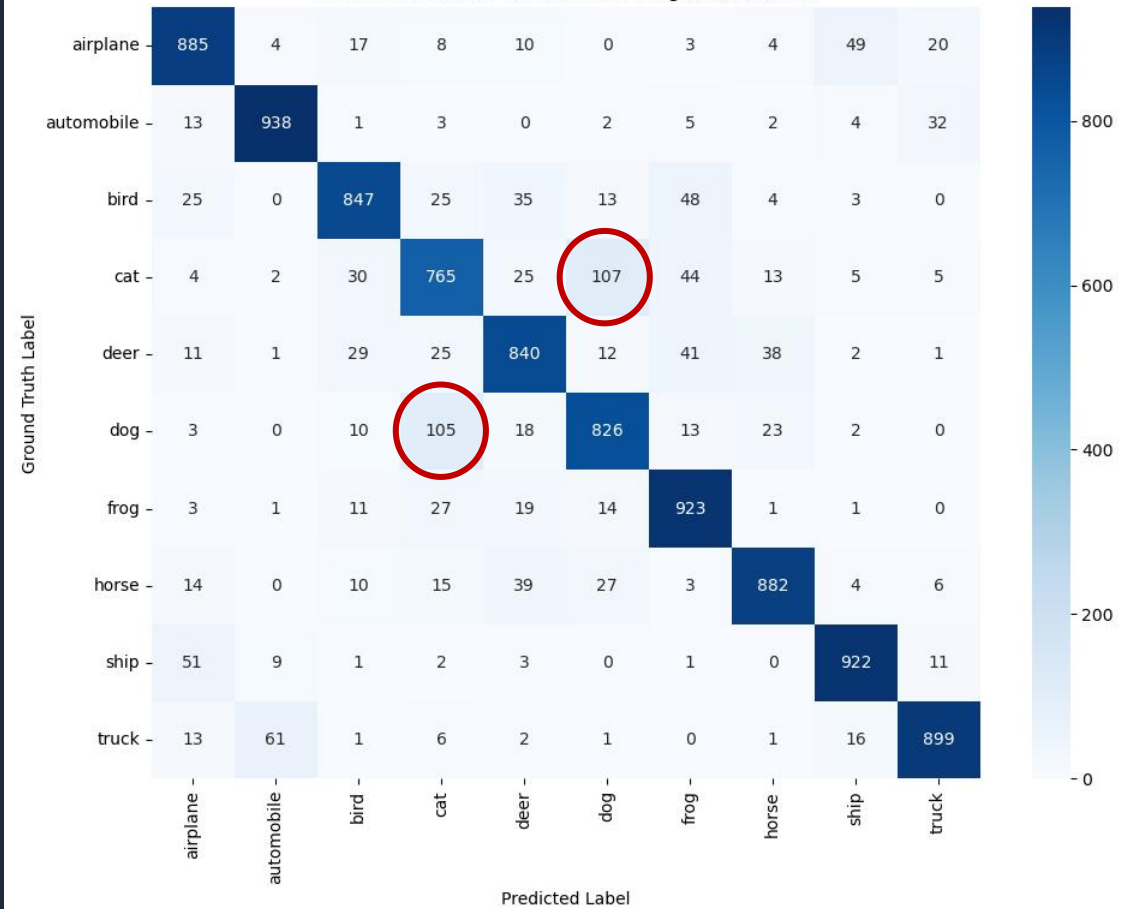
Conclusion: MobileNetV2 achieved 82.5% validation accuracy in its very first epoch. In contrast, it took Custom CNN 18 full epochs of intense training just to cross the 75% mark. This proves that MobileNetV2 didn't just learn faster; it arrived already knowing how to extract complex features, requiring only minor classification adjustments to immediately master our specific dataset.

Confusion Matrix of Test Set

Confusion Matrix - Custom CNN from Scratch



Confusion Matrix - Transfer Learning MobileNetV2



Final Performance Metrics

Metric	Custom CNN (From Scratch)	MobileNetV2 (Transfer Learning)
Total Epochs Run	25/30 Epochs (Early Stopped)	20 Epochs (Completed)
Total Training Time (CPU)	~22 minutes	~76 minutes
Final Test Accuracy	76%	87%
Best Class Performance	Automobile (90%)	Automobile (93%)
Weakest Class Performance	Cat (61%)	Cat (77%)

Project Challenges and Solutions

Challenge	Solution
Overfitting and Class Confusion (Custom CNN Focus)	• Data Augmentation (random crops and horizontal flips) • Dropout regularization (up to 45%) to the dense layers • Implemented an Early Stopping safety net
Strict Hardware Limitations (Custom Transfer Learning Focus)	• Selected MobileNetV2 instead of ResNet or VGG16 • Froze over 2.2 million parameters in the pre-trained feature extractor, reducing trainable weights to just 12,810 • Resized the photos to 128x128 instead of 224x224

Thank You !